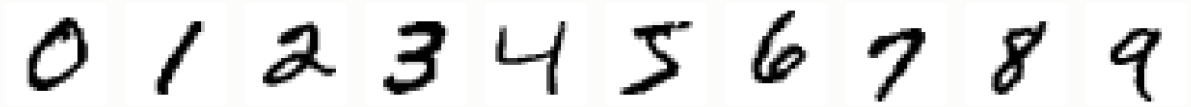
ASSIGNMENT #4.1 · AI HW FALL 2026

MNIST recognition with small neural networks

Shallow MLP vs. CNN vs. Transformer encoder, and what data augmentation buys each of them. Trained on the train split, tested on the test split.



Shengjie Min

Code, checkpoints and test results: https://github.com/Minsj2015/ai-hw-Fall-2026-nn

SETUP

Train on train, test on test

Train split: 60,000 images
55,000 used to fit the weights (with or without augmentation)

5k val

The 5,000 validation images pick the best epoch. Nothing about training or model selection ever reads the test set.

Test split
10,000

Loaded only by `src/test.py`, once per final checkpoint. Results are saved per image in `results/<run>/predictions.csv`.

- Same data pipeline, optimizer and schedule for every model, so differences come from the architecture and the augmentation.
- Each architecture is trained without augmentation and with augmentation (MLP and CNN also with an affine-only ablation): 8 runs in total.

Shared recipe	
Optimizer	AdamW
Schedule	One-cycle LR, 15% warm-up
Batch size	128
Loss	Cross-entropy, smoothing 0.1
Grad clipping	1.0
Epochs	MLP 30 · CNN 12 · ViT 20
Hardware	Laptop CPU (4 cores), no GPU
Seed	0 (fixed split + init)

MODELS

Three architectures, three different assumptions

Shallow MLP

407,050 parameters

Every pixel is an independent input. No notion of locality or shift.

Image 28×28
flattened to 784

↓

Linear 784 → 512
ReLU, dropout 0.2

↓

Linear 512 → 10

CNN

140,458 parameters

Local 3×3 filters shared across positions: built-in locality and shift tolerance.

2× Conv 3×3, 32 ch
BatchNorm + ReLU

↓

MaxPool → 14×14
2× Conv 3×3, 64 ch

↓

MaxPool → 7×7
Conv 3×3, 128 ch

↓

Global avg pool
dropout 0.3, Linear → 10

Transformer encoder (ViT)

139,018 parameters

Image as a sequence of patches; attention learns which patches matter.

4×4 patches → 49 tokens
linear embed to 64-d

↓

+ [CLS] token
+ learned positions

↓

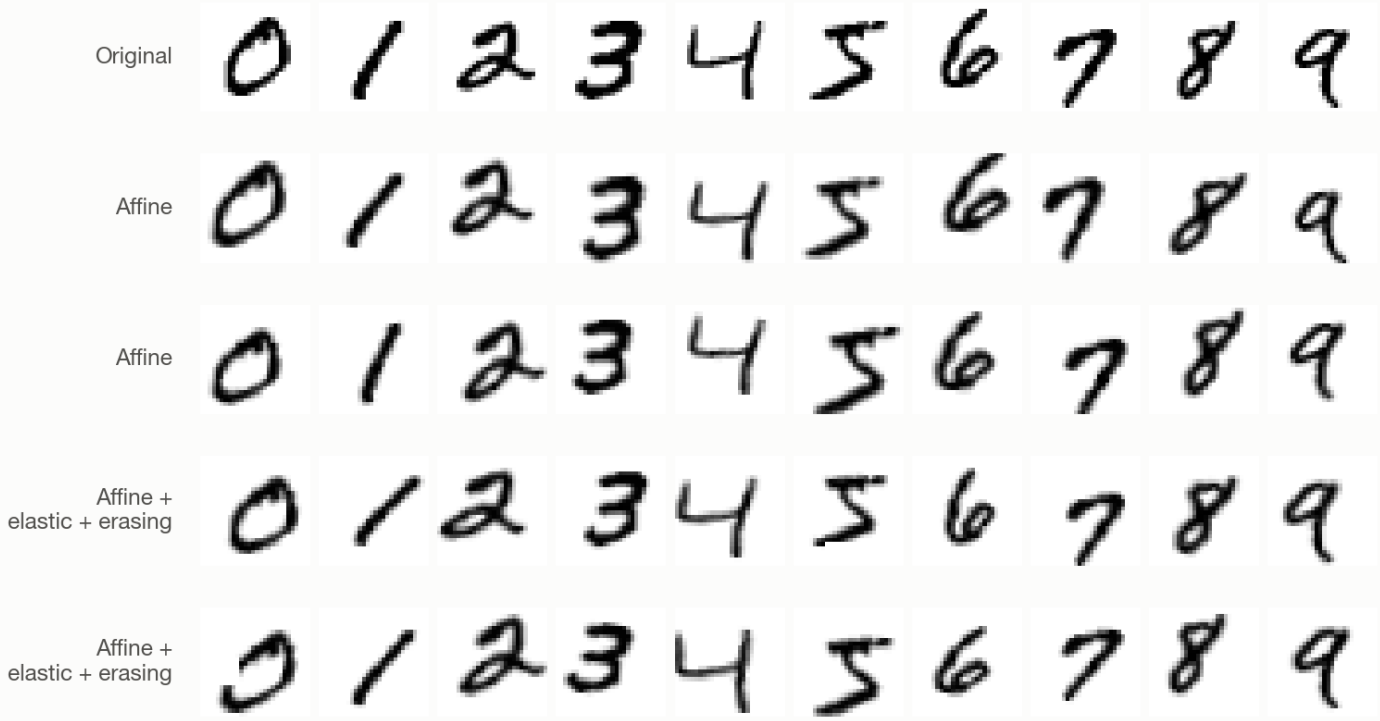
4× encoder layer
4-head self-attn + MLP 128

↓

[CLS] → LayerNorm
Linear → 10

DATA

Augmentation: new training images every epoch



- **Affine**: rotate ±10°, scale 0.9–1.1, shear ±10°, shift ±10% (2.8 px).
- **Full** = affine + **elastic** distortion (Simard et al. 2003: smoothed random displacement, like hand tremor) + **random erasing** of a small patch (30% of images).
- No flips: a mirrored 2 or 7 is not a digit.
- Done on the whole batch at once with `affine_grid` / `grid_sample`: about 12 ms per 128 images on the CPU.
- Applied to training batches only, never to validation or test images.

RESULTS

Results on the 10,000 test images

MLP · affine

99.04%

96 errors. Other settings: no augmentation 98.59%, full augmentation 98.88%.

CNN · no augmentation

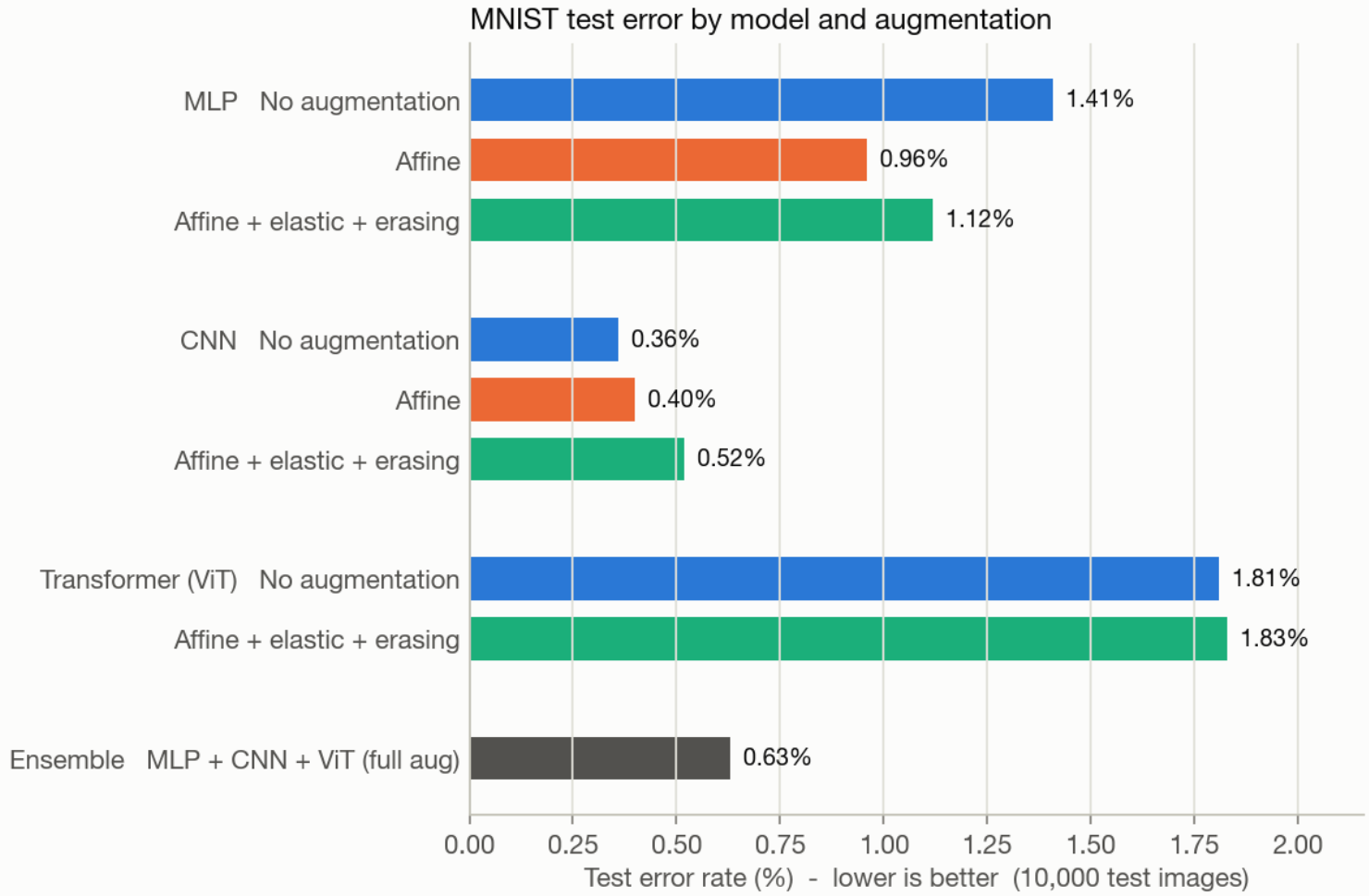
99.64%

36 errors. Other settings: affine 99.60%, full augmentation 99.48%.

Transformer · full augmentation

98.17%

183 errors. Other settings: no augmentation 98.19%.

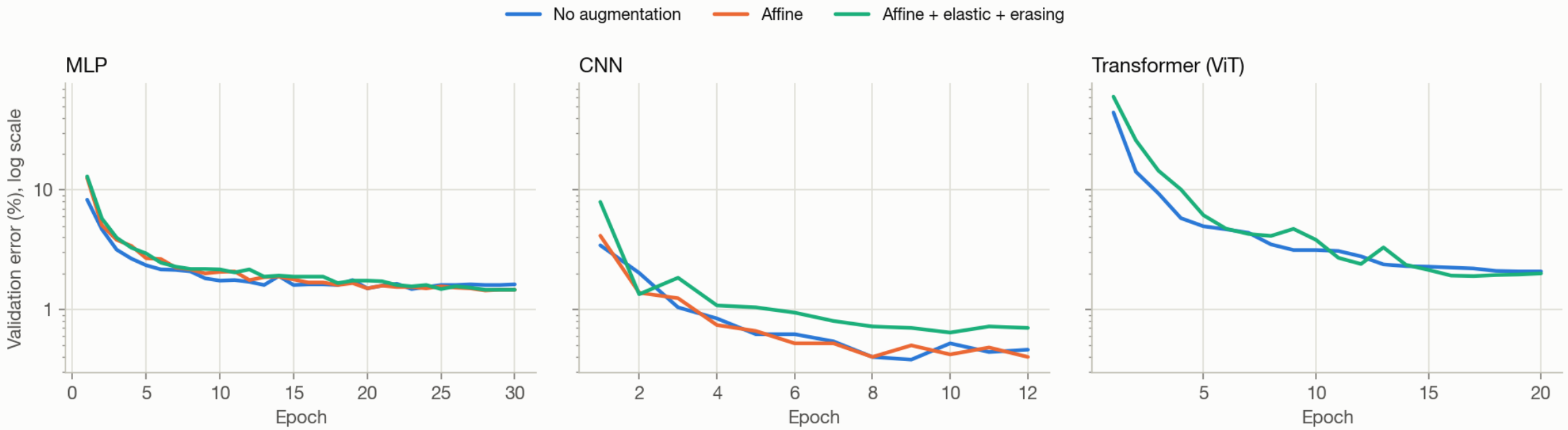


Each model's setting is the one with the best validation accuracy, so the test set is not used to choose it.

Averaging the probabilities of the three fully augmented models (chosen in advance) gives 99.37% (63 errors).

RESULTS

Training curves (validation error per epoch)



- **MLP** without augmentation memorises the training set (99.98% train vs 98.38% validation in the last epoch). Augmentation makes memorising impossible.
- **CNN** converges in about 8 epochs. With full augmentation it is still behind on clean images after 12 epochs: the distortions make the task harder and need a longer schedule.
- **ViT** learns slowest and is still improving at epoch 20. With full augmentation it fits only 96.5% of its distorted training batches, so it is under-trained, not over-fitted.

STATISTICS

Are the differences real?

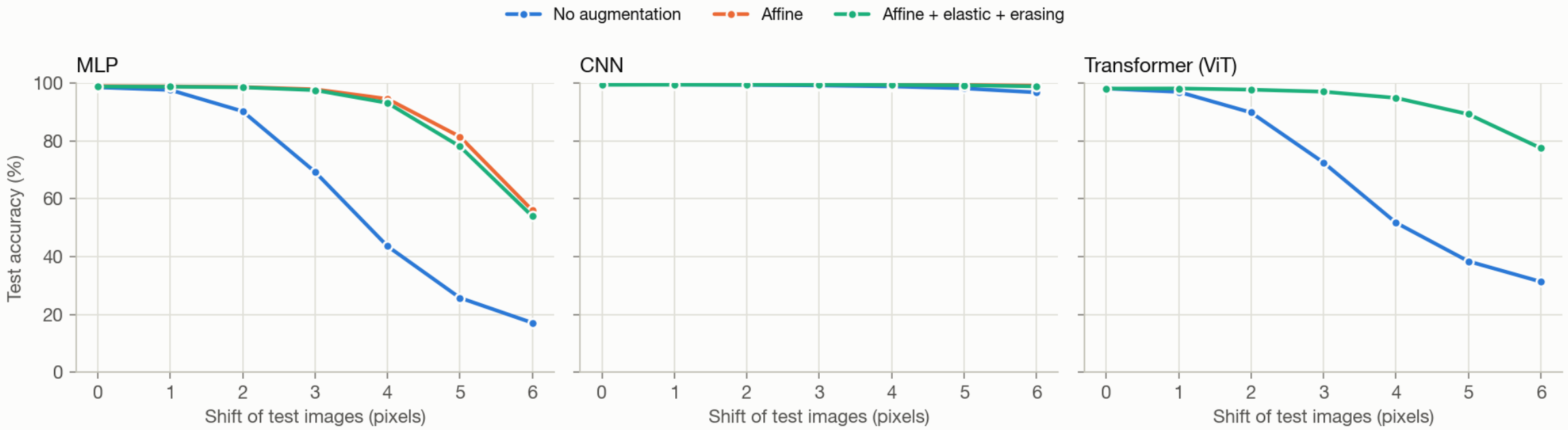
Each pair of models is compared on the same 10,000 test images with an exact McNemar test: only the images where exactly one of the two models is right carry information.

Comparison	Errors A → B	Only A right	Only B right	p-value	α = 0.05
mlp_none vs mlp_affine	141 → 96	25	70	4.3e-06	significant
mlp_none vs mlp_full	141 → 112	38	67	0.006	significant
cnn_none vs cnn_affine	36 → 40	18	14	0.6	not significant
cnn_none vs cnn_full	36 → 52	30	14	0.023	significant
vit_none vs vit_full	181 → 183	118	116	0.95	not significant
mlp_full vs cnn_full	112 → 52	25	85	7.8e-09	significant
cnn_full vs vit_full	52 → 183	154	23	5.4e-25	significant
mlp_full vs vit_full	112 → 183	134	63	4.7e-07	significant
cnn_full vs ensemble	52 → 63	30	19	0.15	not significant

Shaded rows: difference is statistically significant at α = 0.05. With one training seed per configuration, differences of a few errors are within run-to-run noise.

ROBUSTNESS

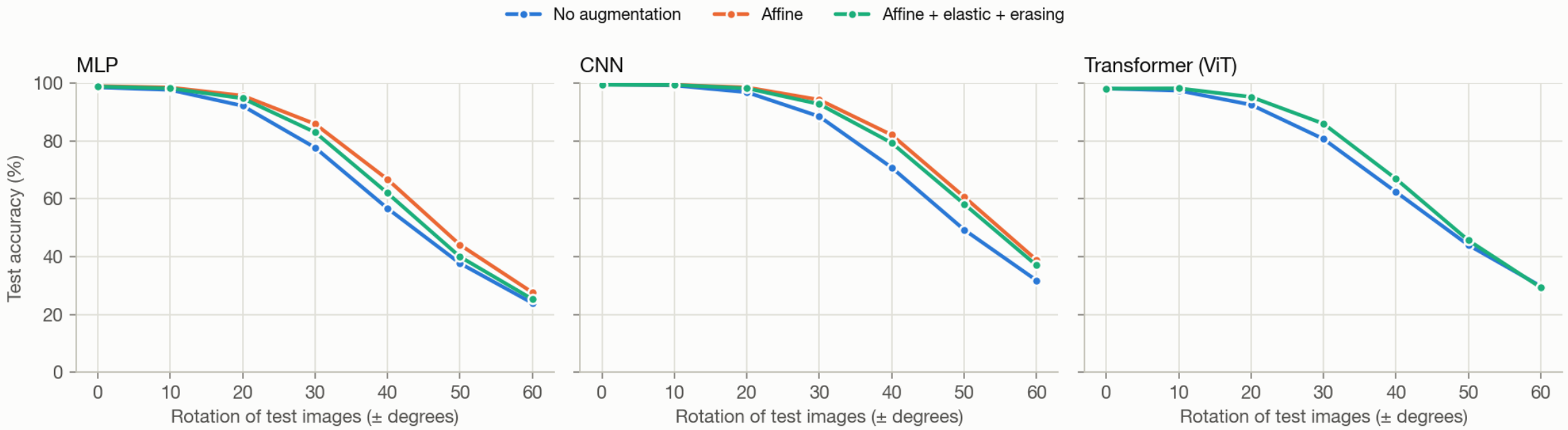
Shifted test digits: augmentation matters most here



- Moving digits 4 px: **MLP 43.6% → 94.6%** with affine augmentation, **ViT 51.8% → 95.0%** with full augmentation.
- The CNN keeps 98.9% without augmentation: convolution + pooling build shift tolerance into the architecture. The MLP and ViT have to learn it from the data.
- That is what augmentation is for: it teaches a model the invariance its architecture is missing.

ROBUSTNESS

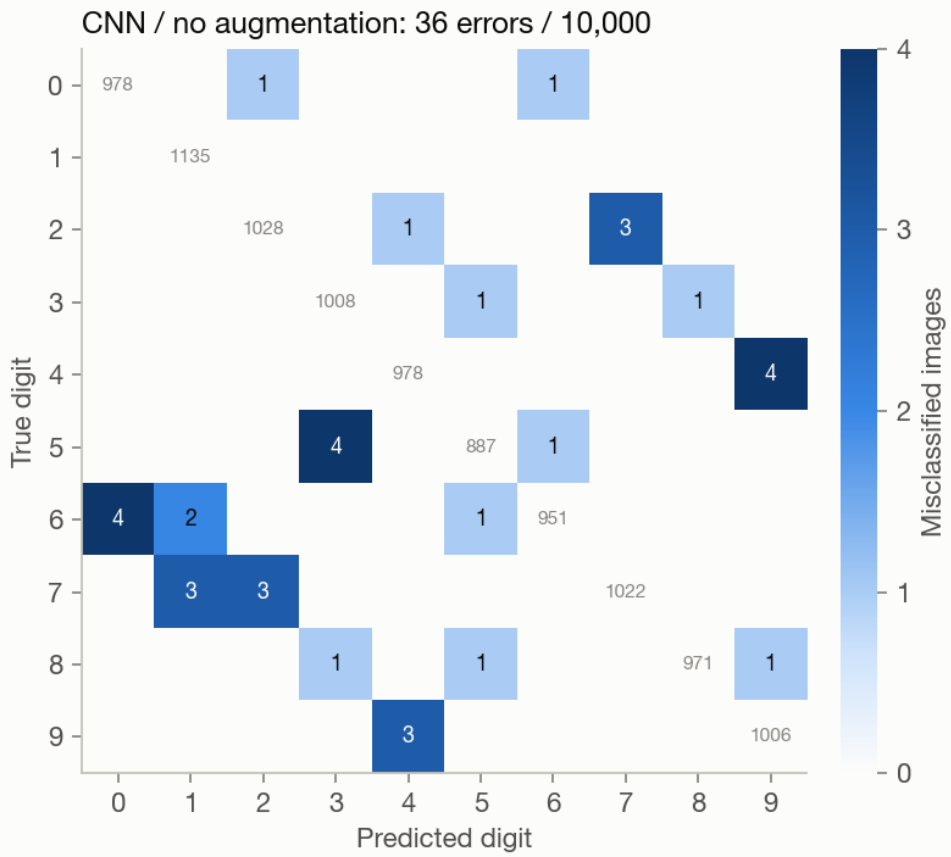
Rotated test digits

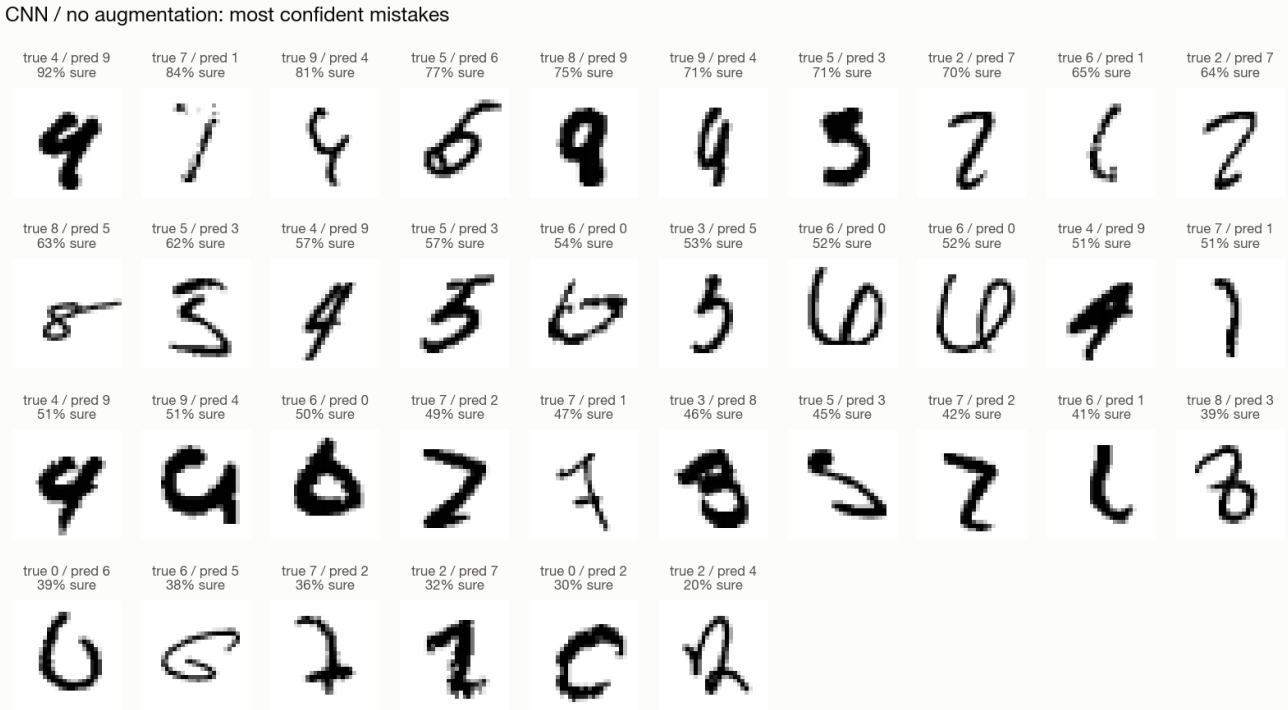


- Training rotations were only ±10°, so every model degrades beyond about 20°.
- At ±30°: CNN 88.5% → 94.3%, MLP 77.8% → 85.9% (affine), ViT 80.7% → 86.0% (full).
- Augmentation does not extrapolate: at ±60° every model is below 39%, because a rotated 6 and a 9 are genuinely ambiguous.

ERRORS

What is still wrong: CNN, no augmentation





Best single model by validation accuracy: **36 errors in 10,000**. Most frequent confusions: 6 read as 0 (4×), 5 read as 3 (4×), 4 read as 9 (4×). Many of the remaining mistakes are ambiguous even for a person.

SUMMARY

Takeaways

- **Architecture matters most.** The CNN (140k parameters) makes 36–52 errors; the MLP, with 3× more parameters, makes 96–141. Locality and weight sharing are the right prior for images.
- **Augmentation adds test % where the architecture lacks the invariance**: MLP 98.59% → 99.04% ($p < 0.001$). For the CNN it did not help clean accuracy at 12 epochs (99.64% vs 99.60%, not significant; the strongest setting was worse).
- **Its biggest payoff is robustness**: on 4-px-shifted digits the MLP goes 44% → 95% and the ViT 52% → 95%.
- **A small Transformer is data- and compute-hungry**: 98.19% after 20 CPU epochs, still improving. Without convolution's built-in priors it has to learn locality from scratch.
- **Ensembling unequal models did not help**: 99.37% vs 99.48% for the CNN alone (not significant). An average is only as good as its members are comparable.

CODE

Reproduce it

```
git clone https://github.com/Minsj2015/ai-hw-Fall-2026-nn.git
cd ai-hw-Fall-2026-nn && pip install -r requirements.txt

# evaluate the committed checkpoints on the test split (~12 min on a CPU)
python src/test.py

# train one model (train split only), then test it
python src/train.py --model cnn --aug full
python src/test.py --ckpt checkpoints/cnn_full.pt

# everything: 8 training runs + testing (~2 h on a 4-core laptop CPU)
./run_all.sh
```

`src/data.py` loading + augmentation · `src/models.py` MLP / CNN / ViT · `src/train.py` · `src/test.py` · `results/` metrics, per-image predictions, figures